

White-Label Multi-App Architecture Guide

A comprehensive guide for building multiple branded mobile applications from a single React Native codebase

Table of Contents

- 1. [Overview](#)
- 2. [Architecture Principles](#)
- 3. [Project Structure](#)
- 4. [Multi-App Strategy](#)
- 5. [Design System & Theming](#)
- 6. [Feature Management](#)
- 7. [Environment Configuration](#)
- 8. [CI/CD Pipeline](#)
- 9. [Feature Flags](#)
- 10. [Build & Deployment](#)
- 11. [Best Practices](#)

Overview

This architecture enables building **multiple branded mobile applications** (iOS & Android) from a **single React Native codebase** using a monorepo structure. It supports:

- ✔ Multiple white-label apps with distinct branding
- ✔ Shared features across all apps
- ✔ App-specific features and customizations
- ✔ Centralized design system with brand-specific tokens
- ✔ Environment-based configuration (dev, staging, production)
- ✔ Feature flags for controlled rollouts
- ✔ Automated CI/CD pipelines

Key Benefits

Benefit	Description
Code Reusability	70-80% code shared across all branded apps
Faster Time-to-Market	New brands can be launched quickly
Consistency	Shared components ensure consistent UX patterns
Maintainability	Bug fixes and features propagate to all apps
Scalability	Easy to add new brands or features
Cost Efficiency	Single development team manages multiple apps

Architecture Principles

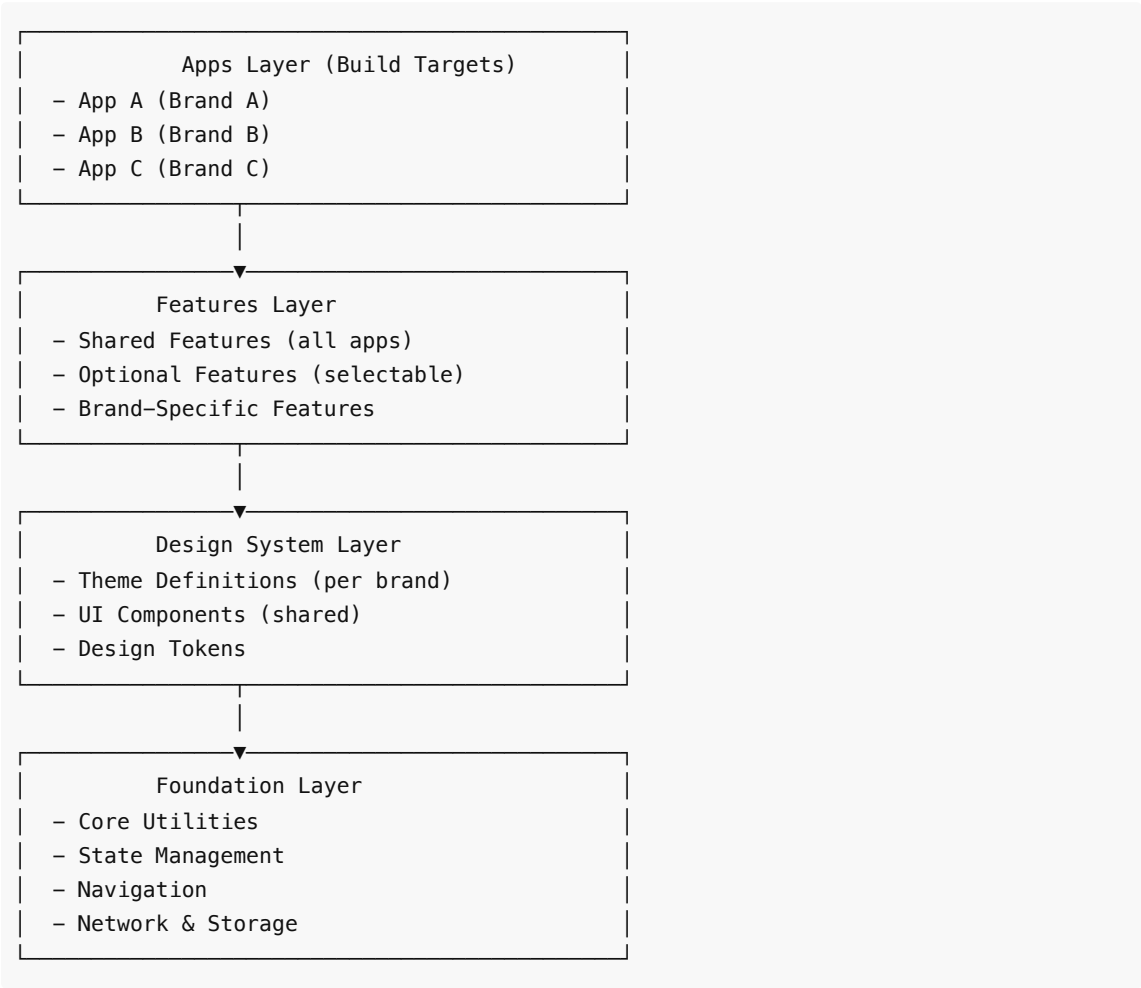
1. Monorepo Structure

- **Technology:** Yarn Workspaces
- **Package Manager:** Yarn (v1.22+)
- **Benefits:** Shared dependencies, atomic commits, single source of truth

2. Feature-Based Modularity

- Each feature is an isolated package
- Features declare their dependencies explicitly
- Features can be enabled/disabled per app

3. Separation of Concerns



Technology Stack

Core Framework & Languages

Category	Technology	Version	Purpose
Mobile Framework	React Native	0.78.2+	Cross-platform mobile development
App Languages	TypeScript	5.0+	React Native app code
iOS Native	Swift	5.0+	iOS native modules

Android Native	Kotlin	1.9+	Android native modules
Package Manager	Yarn Workspaces	1.22+	Monorepo dependency management

Build Tools & Infrastructure

Category	Technology	Purpose
Android Build	Gradle, Android Studio	Android compilation & packaging
iOS Build	Xcode, CocoaPods	iOS compilation & packaging
CI/CD	GitHub Actions	Automated build, test, deploy
Code Quality	ESLint, Prettier, TypeScript	Linting and formatting

Architecture & State

Category	Technology	Purpose
Design System	Monorepo Package (Atomic Design)	UI components + brand themes
State Management	MobX	Local reactive state
Server State	React Query	API caching & synchronization
Navigation	React Navigation	Screen routing
Storage	MMKV, AsyncStorage	Local data persistence

Backend & Cloud Services

Category	Technology	Recommendation	Purpose
Content Management	Any Headless CMS	Contentstack, Contentful, Strapi	Dynamic content delivery
Remote Config	Firebase Remote Config	-	Runtime feature flags
Authentication	Okta / Auth0 / Cognito	As per requirements	User authentication
API Layer	GraphQL / REST	-	Backend communication

Monitoring & Analytics

Category	Technology	Recommendation	Purpose
Crash Reporting	Firebase Crashlytics / Sentry	Sentry Recommended	Real-time crash detection
Performance Monitoring	Firebase Performance / Sentry	Sentry Recommended	App performance insights


```

├── android/                                # Android native code
│   └── app/src/
│       ├── brandA/                        # Brand A build variant
│       ├── brandB/                        # Brand B build variant
│       └── brandC/                        # Brand C build variant
├── ios/                                    # iOS native code
│   ├── mobile.xcodeproj/
│   └── mobile.xcworkspace/
├── src/
│   ├── App.tsx                            # Main app component
│   └── state/                             # Global state
├── scripts/
│   ├── prepare-env/                      # Environment setup scripts
│   └── prepare-env.mjs
└── package.json

├── packages/
│   ├── design-system/                    # UI components + brand themes
│   │   ├── src/
│   │   │   ├── atoms/                    # Box, Text, Button
│   │   │   ├── molecules/                # SectionHeader, InfoCard
│   │   │   ├── organisms/                # Header, Carousel
│   │   │   └── theme/                    # ThemeProvider, hooks
│   │   └── themes/                       # Brand-specific themes
│   │       ├── brandA/
│   │       │   ├── colors.ts
│   │       │   ├── typography.ts
│   │       │   └── logos.ts
│   │       ├── brandB/
│   │       └── brandC/
│   ├── common/                           # Core shared functionality
│   │   ├── core/
│   │   │   ├── src/
│   │   │   │   ├── services/             # API services
│   │   │   │   ├── models/              # Data models
│   │   │   │   ├── utils/               # Utilities
│   │   │   │   └── navigation/           # Navigation setup
│   │   │   └── package.json
│   ├── toolkit/                          # Foundation utilities
│   │   ├── core/
│   │   │   ├── src/
│   │   │   │   ├── types/               # Shared TypeScript types
│   │   │   │   ├── storage/             # Storage (MMKV, AsyncStorage)
│   │   │   │   └── validation/           # Form validation
│   │   │   └── package.json
│   ├── shared-*/                          # Cross-app shared features
│   │   ├── shared-auth/                  # Authentication
│   │   ├── shared-profile/               # User profile
│   │   └── shared-home/                  # Home screen

```

```

├── shared-onboarding/      # Onboarding flows
├── shared-localization/    # i18n/l10n
├── feat-*/                 # Optional features
│   ├── feat-loyalty/      # Loyalty program
│   ├── feat-payments/     # Payment processing
│   ├── feat-store-locator/ # Store finder
│   ├── feat-promotions/   # Deals & offers
│   └── feat-notifications/ # Push notifications
├── brand-specific/        # Brand-specific features
│   ├── brandA-rewards/    # Brand A exclusive
│   ├── brandB-marketplace/ # Brand B exclusive
│   └── brandC-gaming/     # Brand C exclusive
├── docs/
│   ├── architecture/
│   ├── styleguide/
│   └── api/
├── scripts/
│   ├── build/
│   └── deploy/
├── .github/
│   └── workflows/         # CI/CD pipelines
│       ├── build-brandA.yml
│       ├── build-brandB.yml
│       └── build-brandC.yml
├── package.json           # Root workspace config
├── tsconfig.base.json     # TypeScript base config
├── babel.config.js
└── README.md

```

Multi-App Strategy

Build Variants Approach

Each branded app is created as a **build variant** (Android) or **scheme** (iOS) of the same React Native project.

Android Build Variants

File: apps/mobile/android/app/build.gradle

```

android {
    flavorDimensions "brand"

    productFlavors {
        brandA {
            dimension "brand"
            applicationId "com.company.branda"
        }
    }
}

```

```

        manifestPlaceholders = [appName: "Brand A"]
        resValue "string", "app_name", "Brand A"
    }

    brandB {
        dimension "brand"
        applicationId "com.company.brandb"
        manifestPlaceholders = [appName: "Brand B"]
        resValue "string", "app_name", "Brand B"
    }

    brandC {
        dimension "brand"
        applicationId "com.company.brandc"
        manifestPlaceholders = [appName: "Brand C"]
        resValue "string", "app_name", "Brand C"
    }
}

buildTypes {
    debug { /* ... */ }
    release { /* ... */ }
}
}

```

iOS Schemes

Each brand has its own **Xcode scheme**:

1. Scheme Configuration:

- mobile-brandA-debug
- mobile-brandA-release
- mobile-brandB-debug
- etc.

2. Info.plist per Brand:

- brandA-Info.plist
- brandB-Info.plist
- brandC-Info.plist

3. Bundle Identifiers:

- com.company.branda
- com.company.brandb
- com.company.brandc

Design System & Theming

Overview: Monorepo Package + Atomic Design

The design system is a **workspace package within the monorepo** following **atomic design methodology**, with brand themes colocated inside the same package.

Atomic Design Hierarchy:

```
Atoms (Smallest units)
↓
Box, Text, Button, Icon, Input, Divider
↓
Molecules (Atom combinations)
↓
SectionHeader, InfoCard, Pricing
↓
Organism (Complex components)
↓
Header, BottomSheet, Carousel, ProductCard
↓
Templates (Page layouts – in app code)
↓
HomeTemplate, ProfileTemplate
↓
Pages (Final screens – in app code)
↓
HomeScreen, ProfileScreen
```

Why Atomic Design?

- **Consistency:** Atoms ensure UI consistency across all brands
- **Reusability:** Compose molecules/organisms from atoms
- **Scalability:** Easy to add new components
- **Testability:** Test atoms independently
- **Documentation:** Storybook shows all component states

Why Monorepo Package?

- **Unified Development:** Components and themes evolve together
- **Instant Updates:** No version bumps between design system and apps
- **Type Safety:** Direct TypeScript integration across packages
- **Simplified Workflow:** Single repository for all UI code
- **Easier Testing:** Test components with actual brand themes

Centralized Design Token System

The design system uses a **type-safe token system** that enforces consistency while allowing brand customization.

1. Define Type Schema

File: `packages/toolkit/core/src/types/theme.types.ts`

```
import { type SvgProps } from 'react-native-svg'

export type ColorScheme = {
```



```
// Primary brand colors
PRIMARY: string
SECONDARY: string

// Background colors
BACKGROUND: {
  PRIMARY: string
  SECONDARY: string
  TERTIARY: string
  SUCCESS: string
  WARNING: string
  ERROR: string
  DISABLED: string
  // ... more as needed
}

// Text colors
TEXT: {
  PRIMARY: string
  SECONDARY: string
  DISABLED: string
  INVERSE: string
  SUCCESS: string
  WARNING: string
  ERROR: string
  // ... more as needed
}

// Border colors
BORDER: {
  PRIMARY: string
  FOCUS: string
  DISABLED: string
  ERROR: string
  // ... more as needed
}

// Icon colors
ICON: {
  PRIMARY: string
  SECONDARY: string
  DISABLED: string
  // ... more as needed
}

// Semantic colors
SUCCESS: string
WARNING: string
ERROR: string
INFO: string

// Allow for extensibility
```

```

    [key: string]: string | object
  }

export type TypographyScheme = {
  FONT_FAMILY_REGULAR: string
  FONT_FAMILY_MEDIUM: string
  FONT_FAMILY_BOLD: string
  FONT_FAMILY_BLACK: string

  // Font sizes
  FONT_SIZE_XXS: number
  FONT_SIZE_XS: number
  FONT_SIZE_SM: number
  FONT_SIZE_MD: number
  FONT_SIZE_LG: number
  FONT_SIZE_XL: number
  FONT_SIZE_XXL: number

  [key: string]: string | number | object
}

export type LogoScheme = {
  HEADER_LEFT: React.FC<SvgProps>
  HEADER_RIGHT: React.FC<SvgProps>
  ICON: React.FC<SvgProps>
  FULL: React.FC<SvgProps>
  MAP_MARKER: React.FC<SvgProps>

  LOGO_WIDTH: number
  LOGO_HEIGHT: number

  [key: string]: React.FC<SvgProps> | number
}

export type AppTheme = {
  Colors: ColorScheme
  Typography: TypographyScheme
  Logos: LogoScheme
  Spacing?: SpacingScheme
  BorderRadius?: BorderRadiusScheme
}

```

2. Implement Brand-Specific Themes

File: `packages/design-system/themes/brandA/colors.ts`

```

import { type ColorScheme } from '@packages/toolkit'

export const COLORS: ColorScheme = {
  PRIMARY: '#0066CC', // Brand A blue
  SECONDARY: '#FF6B00', // Brand A orange
}

```

```

BACKGROUND: {
  PRIMARY: '#FFFFFF',
  SECONDARY: '#F5F5F5',
  TERTIARY: '#E0E0E0',
  SUCCESS: '#E8F5E9',
  WARNING: '#FFF3E0',
  ERROR: '#FFE0E0',
  DISABLED: '#FAFAFA',
},

TEXT: {
  PRIMARY: '#212121',
  SECONDARY: '#757575',
  DISABLED: '#BDBDBD',
  INVERSE: '#FFFFFF',
  SUCCESS: '#2E7D32',
  WARNING: '#F57C00',
  ERROR: '#C62828',
},

BORDER: {
  PRIMARY: '#E0E0E0',
  FOCUS: '#0066CC',
  DISABLED: '#EEEEEE',
  ERROR: '#C62828',
},

ICON: {
  PRIMARY: '#424242',
  SECONDARY: '#757575',
  DISABLED: '#BDBDBD',
},

SUCCESS: '#4CAF50',
WARNING: '#FF9800',
ERROR: '#F44336',
INFO: '#2196F3',
}

```

File: packages/design-system/themes/brandB/colors.ts

```

import { type ColorScheme } from '@packages/toolkit'

export const COLORS: ColorScheme = {
  PRIMARY: '#8B5CF6', // Brand B purple
  SECONDARY: '#EC4899', // Brand B pink

  BACKGROUND: {
    PRIMARY: '#FFFFFF',
    SECONDARY: '#F9FAFB',
    TERTIARY: '#F3F4F6',
  },
}

```

```

    SUCCESS: '#ECFDF5',
    WARNING: '#FFFBEF',
    ERROR: '#FEEF2F',
    DISABLED: '#F9FAFB',
  },

  TEXT: {
    PRIMARY: '#111827',
    SECONDARY: '#6B7280',
    DISABLED: '#D1D5DB',
    INVERSE: '#FFFFFF',
    SUCCESS: '#065F46',
    WARNING: '#92400E',
    ERROR: '#991B1B',
  },

  // ... rest of color tokens
}

```

3. Theme Initialization in App

File: apps/mobile/src/App.tsx

```

import {
  ThemeInitializer,
  BrandType,
  ThemeMode
} from '@packages/design-system'
import { getBrand } from './config/environment'
import { BrandATheme } from '@packages/design-system/themes/brandA'
import { BrandBTheme } from '@packages/design-system/themes/brandB'
import { BrandCTheme } from '@packages/design-system/themes/brandC'

// Map brands to their theme implementations
const BRAND_THEMES = {
  [BrandType.BRAND_A]: BrandATheme,
  [BrandType.BRAND_B]: BrandBTheme,
  [BrandType.BRAND_C]: BrandCTheme,
}

const App = () => {
  const currentBrand = getBrand() // Reads from build config/env
  const theme = BRAND_THEMES[currentBrand]

  return (
    <ThemeInitializer
      theme={theme} // Brand-specific theme tokens
      defaultBrand={currentBrand} // Current brand identifier
      defaultMode={ThemeMode.LIGHT} // Light/Dark mode
      persistTheme={true} // Save theme preference
      enableSystemTheme={true} // Follow system dark mode
    >

```

```

        <AppContent />
      </ThemeInitializer>
    )
  }
}

```

4. Using Design System Components

Import atomic components from design system package:

```

import {
  // Atoms
  Box,
  Text,
  Button,
  Icon,

  // Molecules
  SectionHeader,
  InfoCard,

  // Organisms
  Header,
  BottomSheet,
  Carousel,

  // Theme
  useTheme,
  spacing,
} from '@packages/design-system'

const MyScreen = () => {
  const { colors, typography } = useTheme()

  return (
    <Box padding={spacing.lg} backgroundColor={colors.BACKGROUND.PRIMARY}>
      <Header title="Welcome" variant="primary" />

      <SectionHeader
        title="Featured Products"
        onPress={handleViewAll}
      />

      <Carousel data={products}>
        {(item) => <ProductCard product={item} />}
      </Carousel>

      <InfoCard
        icon="info"
        title="Special Offer"
        description="Get 20% off your first purchase!"
      />
    </Box>
  )
}

```

```

    <Button
      variant="primary"
      size="large"
      onPress={handlePress}
    >
      Shop Now
    </Button>
  </Box>
)
}

```

Components automatically use brand theme:

```

// Same code across all brands
<Button variant="primary">Sign In</Button>

// Brand A: Renders with blue background (#0066CC)
// Brand B: Renders with purple background (#8B5CF6)
// Brand C: Renders with green background (brand-specific color)

```

Custom styling with theme tokens:

```

const MyCustomComponent = () => {
  const { colors, typography } = useTheme()

  return (
    <View style={{
      backgroundColor: colors.BACKGROUND.PRIMARY,
      borderColor: colors.BORDER.PRIMARY,
      borderWidth: 1,
      borderRadius: 8,
      padding: spacing.md,
    }}>
      <Text style={{
        color: colors.TEXT.PRIMARY,
        fontFamily: typography.FONT_FAMILY_BOLD,
        fontSize: typography.FONT_SIZE_LG,
      }}>
        Custom Component
      </Text>
    </View>
  )
}

```

Design System: Monorepo Package

The design system is a **workspace package** within the monorepo at `packages/design-system`.

Why Monorepo Package?

- Components and themes evolve together

- Instant updates without version management
- Direct TypeScript type sharing
- Simplified development workflow
- Test components with real brand themes

Package Structure (Atomic Design + Themes):

```

packages/design-system/
├── src/
│   ├── atoms/                                # Basic building blocks
│   │   ├── Box.tsx                          # Layout primitive with spacing
│   │   ├── Text.tsx                         # Typography component
│   │   ├── Button.tsx                      # Button variants
│   │   ├── Icon.tsx                       # Icon component
│   │   ├── Input.tsx                      # Form inputs
│   │   └── Divider.tsx                    # Separator line
│   ├── molecules/                          # Combinations of atoms
│   │   ├── SectionHeader.tsx              # Header with title + optional action
│   │   ├── InfoCard.tsx                  # Card with icon + text content
│   │   ├── Pricing.tsx                   # Price display component
│   │   ├── VisitTile.tsx                 # Visit progress tile
│   │   └── InlineContent.tsx             # Inline content with icon
│   ├── organisms/                         # Complex, feature-rich components
│   │   ├── Header.tsx                    # App header with navigation
│   │   ├── BottomSheet.tsx               # Bottom sheet modal
│   │   ├── Carousel.tsx                  # Image/content carousel
│   │   ├── ProductCard.tsx               # Product display card
│   │   ├── StatusScreen.tsx              # Full-screen status pages
│   │   ├── TierLevels.tsx                # Tier progress display
│   │   └── FeaturedAds.tsx               # Featured ads component
│   ├── theme/                             # Theme system
│   │   ├── ThemeProvider.tsx              # React context provider
│   │   ├── ThemeInitializer.tsx           # App initialization wrapper
│   │   ├── useTheme.ts                    # Hook to access theme
│   │   └── tokens/
│   │       ├── spacing.ts                 # Spacing scale
│   │       └── utils.ts                   # Theme utilities
│   ├── types/                             # TypeScript type definitions
│   │   ├── theme.types.ts                 # Theme-related types
│   │   └── component.types.ts             # Component prop types
│   └── index.ts                           # Public API exports
├── themes/                                # Brand-specific themes
│   ├── brandA/
│   │   ├── colors.ts
│   │   ├── typography.ts
│   │   └── logos.ts

```

```
| | | └─ index.ts
| | └─ brandB/
| |   └─ colors.ts
| |   └─ typography.ts
| |   └─ logos.ts
| |   └─ index.ts
| └─ package.json
| └─ tsconfig.json
└─ README.md
```

Usage in Apps:

```
// apps/brandA/package.json
{
  "dependencies": {
    "@packages/design-system": "*" // Workspace reference
  }
}
```

Feature Management

Feature Package Structure

Each feature is a standalone package with clear boundaries:

```
packages/feat-loyalty/
├─ src/
|   ├─ components/      # UI components
|   ├─ screens/         # Screen components
|   ├─ navigation/      # Navigation config
|   ├─ services/        # API services
|   ├─ hooks/           # Custom hooks
|   ├─ models/          # Data models
|   ├─ utils/           # Utilities
|   └─ index.ts         # Public API
├─ assets/              # Feature-specific assets
├─ __tests__/           # Tests
├─ package.json
└─ README.md
```

Feature Package Configuration

File: packages/feat-loyalty/package.json

```
{
  "name": "@packages/feat-loyalty",
  "version": "1.0.0",
  "private": true,
  "main": "src/index.ts",
}
```



```

"dependencies": {
  "@packages/common": "*",
  "@packages/toolkit": "*",
  "@packages/shared-localization": "*",
  "@react-navigation/native": "^6.0.0",
  "@tanstack/react-query": "^5.0.0"
}
}

```

Feature Module Export

File: packages/feat-loyalty/src/index.ts

```

// Screens
export { LoyaltyHomeScreen } from './screens/LoyaltyHomeScreen'
export { RewardsScreen } from './screens/RewardsScreen'

// Navigation
export { LoyaltyNavigator } from './navigation/LoyaltyNavigator'
export { loyaltyScreens } from './navigation/screens'

// Hooks
export { useLoyaltyPoints } from './hooks/useLoyaltyPoints'
export { useRewards } from './hooks/useRewards'

// Types
export type { LoyaltyState, Reward, Points } from './models/types'

```

Feature Activation Matrix

Create a configuration file that defines which features are active for each brand:

File: apps/mobile/src/config/features.ts

```

export enum BrandType {
  BRAND_A = 'brandA',
  BRAND_B = 'brandB',
  BRAND_C = 'brandC',
}

export enum FeatureFlag {
  LOYALTY = 'loyalty',
  PAYMENTS = 'payments',
  STORE_LOCATOR = 'storeLocator',
  PROMOTIONS = 'promotions',
  MARKETPLACE = 'marketplace',
  GAMING = 'gaming',
  SOCIAL_FEATURES = 'socialFeatures',
}

type FeatureMatrix = {

```

```

    [key in BrandType]: {
      [key in FeatureFlag]?: boolean
    }
  }
}

export const FEATURE_MATRIX: FeatureMatrix = {
  [BrandType.BRAND_A]: {
    [FeatureFlag.LOYALTY]: true,
    [FeatureFlag.PAYMENTS]: true,
    [FeatureFlag.STORE_LOCATOR]: true,
    [FeatureFlag.PROMOTIONS]: true,
    [FeatureFlag.MARKETPLACE]: false,
    [FeatureFlag.GAMING]: false,
  },

  [BrandType.BRAND_B]: {
    [FeatureFlag.LOYALTY]: true,
    [FeatureFlag.PAYMENTS]: true,
    [FeatureFlag.STORE_LOCATOR]: true,
    [FeatureFlag.PROMOTIONS]: false,
    [FeatureFlag.MARKETPLACE]: true,
    [FeatureFlag.GAMING]: false,
  },

  [BrandType.BRAND_C]: {
    [FeatureFlag.LOYALTY]: false,
    [FeatureFlag.PAYMENTS]: true,
    [FeatureFlag.STORE_LOCATOR]: true,
    [FeatureFlag.PROMOTIONS]: true,
    [FeatureFlag.MARKETPLACE]: false,
    [FeatureFlag.GAMING]: true,
  },
}

export const isFeatureEnabled = (
  brand: BrandType,
  feature: FeatureFlag
): boolean => {
  return FEATURE_MATRIX[brand]?.[feature] ?? false
}

```

Dynamic Feature Loading

File: apps/mobile/src/utils/getScreens.ts

```

import { BrandType, FeatureFlag, isFeatureEnabled } from '../config/features'
import { getBrand } from '../config/environment'

export const getScreens = async () => {
  const brand = getBrand() // Get current brand from environment

```

```

const screens = {
  stacks: [],
  tabs: [],
  modals: [],
}

// Always load shared features
const { SharedHomeScreen } = await import('@packages/shared-home')
const { ProfileScreen } = await import('@packages/shared-profile')

screens.tabs.push(
  <Tab.Screen key="home" name="Home" component={SharedHomeScreen} />,
  <Tab.Screen key="profile" name="Profile" component={ProfileScreen} />
)

// Conditionally load optional features
if (isFeatureEnabled(brand, FeatureFlag.LOYALTY)) {
  const { LoyaltyNavigator } = await import('@packages/feat-loyalty')
  screens.tabs.push(
    <Tab.Screen key="loyalty" name="Loyalty" component={LoyaltyNavigator} />
  )
}

if (isFeatureEnabled(brand, FeatureFlag.MARKETPLACE)) {
  const { MarketplaceNavigator } = await import('@packages/brandB-marketplace')
  screens.tabs.push(
    <Tab.Screen key="marketplace" name="Marketplace" component=
{MarketplaceNavigator} />
  )
}

if (isFeatureEnabled(brand, FeatureFlag.GAMING)) {
  const { GamingNavigator } = await import('@packages/brandC-gaming')
  screens.tabs.push(
    <Tab.Screen key="gaming" name="Gaming" component={GamingNavigator} />
  )
}

return screens
}

```

Environment Configuration

Configuration Hierarchy

Each app variant can have different configs for different environments:

Configuration Priority (highest to lowest):

1. {brand}.{environment}.json (e.g., brandA.production.json)
2. {environment}.json (e.g., production.json)

- | | |
|-----------------|---------------------|
| 3. {brand}.json | (e.g., brandA.json) |
| 4. common.json | (base config) |

Configuration Files Structure

```
apps/mobile/scripts/prepare-env/env/
├─ common.json           # Base config for all brands
├─ brandA.json           # Brand A defaults
├─ brandB.json           # Brand B defaults
├─ brandC.json           # Brand C defaults
├─ development.json      # Dev environment
├─ staging.json           # Staging environment
├─ production.json       # Production environment
├─ brandA.development.json # Brand A + Dev
├─ brandA.staging.json   # Brand A + Staging
├─ brandA.production.json # Brand A + Production
├─ brandB.development.json
├─ brandB.staging.json
├─ brandB.production.json
└─ secrets.json          # Encrypted secrets (gitignored)
```

Environment Setup Script

File: apps/mobile/scripts/prepare-env/prepare-env.mjs

```
import * as dotenv from 'dotenv'
import fs from 'fs'
import { dirname, resolve } from 'path'
import { fileURLToPath } from 'url'

const BRANDS = ['brandA', 'brandB', 'brandC']
const ENVIRONMENTS = ['development', 'staging', 'production']

const __filename = fileURLToPath(import.meta.url)
const __dirname = dirname(__filename)

const brand = process.argv[2]?.toLowerCase()
const environment = process.argv[3]?.toLowerCase()

const validateArgs = () => {
  if (!brand || !BRANDS.includes(brand)) {
    console.error(`❌ Invalid brand. Must be one of: ${BRANDS.join(', ')}`)
    process.exit(1)
  }

  if (!environment || !ENVIRONMENTS.includes(environment)) {
    console.error(`❌ Invalid environment. Must be one of: ${ENVIRONMENTS.join(', ')}`)
    process.exit(1)
  }
}
```

```

const loadConfig = async (filename) => {
  const path = resolve(__dirname, 'env', filename)
  try {
    return (await import(path, { assert: { type: 'json' } })).default
  } catch (e) {
    console.warn(`⚠️ Config not found: ${filename}`)
    return {}
  }
}

const generateEnvFile = async () => {
  validateArgs()

  console.log(`🔧 Generating environment for: ${brand} - ${environment}`)

  // Load configs in priority order
  const common = await loadConfig('common.json')
  const brandConfig = await loadConfig(`${brand}.json`)
  const envConfig = await loadConfig(`${environment}.json`)
  const brandEnvConfig = await loadConfig(`${brand}.${environment}.json`)
  const secrets = await loadConfig('secrets.json')

  // Merge configs (later configs override earlier ones)
  const merged = {
    ...common,
    ...brandConfig,
    ...envConfig,
    ...brandEnvConfig,
    ...secrets,
  }

  // Add metadata
  merged.APP_BRAND = brand
  merged.APP_ENVIRONMENT = environment
  merged.BUILD_TIMESTAMP = new Date().toISOString()

  // Convert to .env format
  const envContent = Object.entries(merged)
    .map(([key, value]) => `${key}=${value}`)
    .join('\n')

  // Write to .env file
  const envPath = resolve(__dirname, '../../.env')
  fs.writeFileSync(envPath, envContent)

  console.log(`✅ Environment file created: ${envPath}`)
  console.log(`📦 Brand: ${brand}`)
  console.log(`🌍 Environment: ${environment}`)
}

```

```
generateEnvFile()
```

Example Configuration Files

File: `common.json`

```
{
  "API_TIMEOUT": "30000",
  "ENABLE_ANALYTICS": "true",
  "LOG_LEVEL": "info",
  "CACHE_TTL": "3600"
}
```

File: `brandA.json`

```
{
  "API_BASE_URL": "https://api.branda.com",
  "APP_NAME": "Brand A",
  "THEME": "brandA",
  "PRIMARY_COLOR": "#0066CC",
  "SUPPORT_EMAIL": "support@branda.com"
}
```

File: `production.json`

```
{
  "API_BASE_URL": "https://api.production.com",
  "ENABLE_DEV_TOOLS": "false",
  "LOG_LEVEL": "error",
  "SENTRY_DSN": "https://sentry.io/..."
}
```

File: `brandA.production.json`

```
{
  "API_BASE_URL": "https://api.branda.com/v1",
  "ANALYTICS_ID": "GA-BRANDA-PROD",
  "API_KEY": "${BRANDA_PROD_API_KEY}"
}
```

Package.json Scripts

```
{
  "scripts": {
    "env:brandA:dev": "node scripts/prepare-env/prepare-env.mjs brandA development",
    "env:brandA:staging": "node scripts/prepare-env/prepare-env.mjs brandA staging",
    "env:brandA:prod": "node scripts/prepare-env/prepare-env.mjs brandA production",
  }
}
```

```

    "env:brandB:dev": "node scripts/prepare-env/prepare-env.mjs brandB development",
    "env:brandB:staging": "node scripts/prepare-env/prepare-env.mjs brandB staging",
    "env:brandB:prod": "node scripts/prepare-env/prepare-env.mjs brandB production",

    "ios:brandA": "yarn env:brandA:dev && react-native run-ios --scheme=mobile-brandA-debug",
    "ios:brandB": "yarn env:brandB:dev && react-native run-ios --scheme=mobile-brandB-debug",

    "android:brandA": "yarn env:brandA:dev && react-native run-android --mode=brandADebug",
    "android:brandB": "yarn env:brandB:dev && react-native run-android --mode=brandBDebug"
  }
}

```

Feature Flags

Remote Feature Flags System

Implement a feature flag system that allows runtime control of features without app updates.

1. Feature Flag Service

File: `packages/common/core/src/services/FeatureFlagService.ts`

```

import { getValue, fetchAndActivate } from '@react-native-firebase/remote-config'

export enum RemoteFeatureFlag {
  // Feature rollouts
  NEW_CHECKOUT_FLOW = 'new_checkout_flow',
  SOCIAL_SHARING = 'social_sharing',
  DARK_MODE = 'dark_mode',

  // A/B tests
  ONBOARDING_V2 = 'onboarding_v2',
  PRICING_TEST = 'pricing_test',

  // Kill switches
  PAYMENTS_ENABLED = 'payments_enabled',
  CHAT_SUPPORT_ENABLED = 'chat_support_enabled',
}

class FeatureFlagService {
  private cache: Map<string, boolean> = new Map()

  async initialize(): Promise<void> {
    try {
      await fetchAndActivate()
      console.log('✅ Feature flags initialized')
    }
  }
}

```

```

    } catch (error) {
      console.error('❌ Failed to initialize feature flags:', error)
    }
  }

  isEnabled(flag: RemoteFeatureFlag): boolean {
    // Check cache first
    if (this.cache.has(flag)) {
      return this.cache.get(flag)!
    }

    // Get from remote config
    const value = getValue(flag).asBoolean()
    this.cache.set(flag, value)
    return value
  }

  getString(flag: string): string {
    return getValue(flag).asString()
  }

  getNumber(flag: string): number {
    return getValue(flag).asNumber()
  }

  getJSON<T>(flag: string): T | null {
    try {
      return JSON.parse(getValue(flag).asString())
    } catch {
      return null
    }
  }

  clearCache(): void {
    this.cache.clear()
  }
}

export const featureFlagService = new FeatureFlagService()

```

2. Feature Flag Hook

File: packages/common/core/src/hooks/useFeatureFlag.ts

```

import { useEffect, useState } from 'react'
import { featureFlagService, RemoteFeatureFlag } from
'../services/FeatureFlagService'

export const useFeatureFlag = (flag: RemoteFeatureFlag): boolean => {
  const [isEnabled, setIsEnabled] = useState(false)

  useEffect(() => {

```



```

    const checkFlag = () => {
      const enabled = featureFlagService.isEnabled(flag)
      setIsEnabled(enabled)
    }

    checkFlag()
  }, [flag])

  return isEnabled
}

```

3. Usage in Components

```

import { useFeatureFlag } from '@packages/common'
import { RemoteFeatureFlag } from '@packages/common'

const CheckoutScreen = () => {
  const isNewCheckoutEnabled = useFeatureFlag(RemoteFeatureFlag.NEW_CHECKOUT_FLOW)

  if (isNewCheckoutEnabled) {
    return <NewCheckoutFlow />
  }

  return <LegacyCheckoutFlow />
}

```

4. Feature Flag Configuration (Firebase Remote Config)

```

{
  "new_checkout_flow": {
    "defaultValue": {
      "value": "false"
    },
    "conditionalValues": {
      "brandA_production": {
        "value": "true"
      },
      "brandB_beta_users": {
        "value": "true"
      }
    }
  },
  "payments_enabled": {
    "defaultValue": {
      "value": "true"
    }
  },
  "onboarding_v2": {

```

```

    "defaultValue": {
      "value": "false"
    },
    "conditionalValues": {
      "ab_test_group_a": {
        "value": "true"
      }
    }
  }
}
}

```

5. Combined Feature Flags

Combine build-time and runtime feature flags:

```

import { isFeatureEnabled } from './config/features'
import { useFeatureFlag } from '@packages/common'

const MyComponent = () => {
  const brand = getBrand()

  // Build-time check
  const hasLoyaltyModule = isFeatureEnabled(brand, FeatureFlag.LOYALTY)

  // Runtime check
  const isLoyaltyV2Enabled = useFeatureFlag(RemoteFeatureFlag.LOYALTY_V2)

  if (!hasLoyaltyModule) {
    return null // Feature not included in this brand
  }

  if (isLoyaltyV2Enabled) {
    return <LoyaltyV2Screen />
  }

  return <LoyaltyScreen />
}

```

CI/CD Pipeline

GitHub Actions Workflow Structure

```

.github/workflows/
├─ build-brandA.yml      # Brand A builds
├─ build-brandB.yml      # Brand B builds
├─ build-brandC.yml      # Brand C builds
├─ test.yml              # Run tests
├─ lint.yml              # Linting
├─ deploy-staging.yml    # Deploy to staging
└─ deploy-production.yml # Deploy to production

```

Example: Build Workflow for Brand A

File: `.github/workflows/build-brandA.yml`

```
name: Build Brand A

on:
  push:
    branches:
      - main
      - develop
      - 'release/**'
  pull_request:
    branches:
      - main
      - develop

env:
  BRAND: brandA
  NODE_VERSION: '18'
  JAVA_VERSION: '17'
  RUBY_VERSION: '3.2'

jobs:
  # Job 1: Run tests and linting
  test:
    name: Test & Lint
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${ env.NODE_VERSION }
          cache: 'yarn'

      - name: Install dependencies
        run: yarn install --frozen-lockfile

      - name: Run linter
        run: yarn lint

      - name: Run tests
        run: yarn test --coverage

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage/lcov.info
```

```

# Job 2: Build Android
build-android:
  name: Build Android (Brand A)
  runs-on: ubuntu-latest
  needs: test
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${ env.NODE_VERSION }
        cache: 'yarn'

    - name: Setup Java
      uses: actions/setup-java@v4
      with:
        distribution: 'temurin'
        java-version: ${ env.JAVA_VERSION }

    - name: Install dependencies
      run: yarn install --frozen-lockfile

    - name: Setup environment
      run: |
        if [[ "${ github.ref }" == "refs/heads/main" ]]; then
          yarn env:brandA:prod
        elif [[ "${ github.ref }" =~ ^refs/heads/release/ ]]; then
          yarn env:brandA:staging
        else
          yarn env:brandA:dev
        fi

    - name: Decode signing key
      run: |
        echo "${ secrets.ANDROID_KEYSTORE_BASE64 }" | base64 -d >
apps/mobile/android/app/release.keystore

    - name: Build Android APK
      working-directory: apps/mobile/android
      run: ./gradlew assembleBrandARelease
      env:
        KEYSTORE_PASSWORD: ${ secrets.ANDROID_KEYSTORE_PASSWORD }
        KEY_ALIAS: ${ secrets.ANDROID_KEY_ALIAS }
        KEY_PASSWORD: ${ secrets.ANDROID_KEY_PASSWORD }

    - name: Build Android AAB
      working-directory: apps/mobile/android
      run: ./gradlew bundleBrandARelease

    - name: Upload APK artifact

```

```

    uses: actions/upload-artifact@v4
    with:
      name: brandA-android-apk
      path: apps/mobile/android/app/build/outputs/apk/brandA/release/*.apk

  - name: Upload AAB artifact
    uses: actions/upload-artifact@v4
    with:
      name: brandA-android-aab
      path: apps/mobile/android/app/build/outputs/bundle/brandARelease/*.aab

```

Job 3: Build iOS

```

build-ios:
  name: Build iOS (Brand A)
  runs-on: macos-14
  needs: test
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${ env.NODE_VERSION }
        cache: 'yarn'

    - name: Setup Ruby
      uses: ruby/setup-ruby@v1
      with:
        ruby-version: ${ env.RUBY_VERSION }
        bundler-cache: true
        working-directory: apps/mobile/ios

    - name: Install dependencies
      run: yarn install --frozen-lockfile

    - name: Setup environment
      run: |
        if [[ "${ github.ref }" == "refs/heads/main" ]]; then
          yarn env:brandA:prod
        elif [[ "${ github.ref }" =~ ^refs/heads/release/ ]]; then
          yarn env:brandA:staging
        else
          yarn env:brandA:dev
        fi

    - name: Install CocoaPods
      working-directory: apps/mobile/ios
      run: pod install

    - name: Import signing certificate
      uses: apple-actions/import-codesign-certs@v2

```

```

    with:
      p12-file-base64: ${ secrets.IOS_CERTIFICATE_P12_BASE64 }
      p12-password: ${ secrets.IOS_CERTIFICATE_PASSWORD }

- name: Download provisioning profile
  run: |
    mkdir -p ~/Library/MobileDevice/Provisioning\ Profiles
    echo "${ secrets.IOS_PROVISIONING_PROFILE_BASE64 }" | base64 -d >
~/Library/MobileDevice/Provisioning\ Profiles/brandA.mobileprovision

- name: Build iOS app
  working-directory: apps/mobile/ios
  run: |
    xcodebuild -workspace mobile.xcworkspace \
      -scheme mobile-brandA-release \
      -configuration Release \
      -archivePath build/BrandA.xcarchive \
      archive

- name: Export IPA
  working-directory: apps/mobile/ios
  run: |
    xcodebuild -exportArchive \
      -archivePath build/BrandA.xcarchive \
      -exportPath build \
      -exportOptionsPlist ExportOptions.plist

- name: Upload IPA artifact
  uses: actions/upload-artifact@v4
  with:
    name: brandA-ios-ipa
    path: apps/mobile/ios/build/*.ipa

# Job 4: Deploy to App Center (Staging)
deploy-appcenter:
  name: Deploy to App Center
  runs-on: ubuntu-latest
  needs: [build-android, build-ios]
  if: github.ref == 'refs/heads/develop'
  steps:
    - name: Download Android APK
      uses: actions/download-artifact@v4
      with:
        name: brandA-android-apk
        path: ./artifacts/android

    - name: Download iOS IPA
      uses: actions/download-artifact@v4
      with:
        name: brandA-ios-ipa
        path: ./artifacts/ios

```

```

- name: Deploy Android to App Center
  uses: wzieba/AppCenter-Github-Action@v1
  with:
    appName: YourOrg/BrandA-Android
    token: ${ secrets.APPCENTER_TOKEN }
    group: Internal
    file: ./artifacts/android/*.apk
    releaseNotes: |
      Build: ${ github.run_number }
      Branch: ${ github.ref_name }
      Commit: ${ github.sha }

- name: Deploy iOS to App Center
  uses: wzieba/AppCenter-Github-Action@v1
  with:
    appName: YourOrg/BrandA-iOS
    token: ${ secrets.APPCENTER_TOKEN }
    group: Internal
    file: ./artifacts/ios/*.ipa
    releaseNotes: |
      Build: ${ github.run_number }
      Branch: ${ github.ref_name }
      Commit: ${ github.sha }

# Job 5: Deploy to Google Play (Production)
deploy-google-play:
  name: Deploy to Google Play
  runs-on: ubuntu-latest
  needs: build-android
  if: github.ref == 'refs/heads/main'
  steps:
    - name: Download Android AAB
      uses: actions/download-artifact@v4
      with:
        name: brandA-android-aab
        path: ./artifacts

    - name: Deploy to Google Play
      uses: r0adkll/upload-google-play@v1
      with:
        serviceAccountJsonPlainText: ${ secrets.GOOGLE_PLAY_SERVICE_ACCOUNT }
        packageName: com.company.branda
        releaseFiles: ./artifacts/*.aab
        track: internal
        status: completed

# Job 6: Deploy to App Store (Production)
deploy-app-store:
  name: Deploy to App Store
  runs-on: macos-14
  needs: build-ios
  if: github.ref == 'refs/heads/main'

```

```

steps:
  - name: Download iOS IPA
    uses: actions/download-artifact@v4
    with:
      name: brandA-ios-ipa
      path: ./artifacts

  - name: Upload to App Store Connect
    uses: apple-actions/upload-testflight-build@v1
    with:
      app-path: ./artifacts/*.ipa
      issuer-id: ${ secrets.APPSTORE_ISSUER_ID }
      api-key-id: ${ secrets.APPSTORE_API_KEY_ID }
      api-private-key: ${ secrets.APPSTORE_API_PRIVATE_KEY }

```

Secrets Configuration

Required GitHub Secrets:

```

# Android
ANDROID_KEYSTORE_BASE64      # Base64 encoded keystore
ANDROID_KEYSTORE_PASSWORD    # Keystore password
ANDROID_KEY_ALIAS            # Key alias
ANDROID_KEY_PASSWORD         # Key password

# iOS
IOS_CERTIFICATE_P12_BASE64    # Base64 encoded certificate
IOS_CERTIFICATE_PASSWORD      # Certificate password
IOS_PROVISIONING_PROFILE_BASE64 # Base64 encoded provisioning profile

# App Center
APPCENTER_TOKEN               # App Center API token

# Google Play
GOOGLE_PLAY_SERVICE_ACCOUNT   # Service account JSON

# App Store
APPSTORE_ISSUER_ID            # App Store Connect issuer ID
APPSTORE_API_KEY_ID           # API key ID
APPSTORE_API_PRIVATE_KEY      # Private key

```

Fastlane Integration

File: `apps/mobile/ios/fastlane/Fastfile`

```

default_platform(:ios)

platform :ios do
  desc "Build and deploy Brand A to TestFlight"
  lane :deploy_brandA do |options|
    # Setup

```



```

setup_ci if ENV['CI']

# Match certificates
match(
  type: "appstore",
  app_identifier: "com.company.branda",
  readonly: true
)

# Build
build_app(
  workspace: "mobile.xcworkspace",
  scheme: "mobile-brandA-release",
  export_method: "app-store",
  output_directory: "./build"
)

# Upload to TestFlight
upload_to_testflight(
  app_identifier: "com.company.branda",
  skip_waiting_for_build_processing: true,
  changelog: options[:changelog] || "New build"
)
end
end

```

Build & Deployment

Build Versioning Strategy

Implement semantic versioning with build numbers:

Version format: MAJOR.MINOR.PATCH (BUILD_NUMBER)
 Example: 2.5.1 (450)

- MAJOR: Breaking changes
- MINOR: New features
- PATCH: Bug fixes
- BUILD_NUMBER: Auto-incremented per build

Automated Version Bumping

File: scripts/version-bump.sh

```

#!/bin/bash

TYPE=$1 # major, minor, or patch

if [ -z "$TYPE" ]; then
  echo "Usage: ./version-bump.sh [major|minor|patch]"
  exit 1

```

```

fi

# Bump version in package.json
npm version $TYPE --no-git-tag-version

# Get new version
NEW_VERSION=$(node -p "require('./package.json').version")

# Update iOS version
/usr/libexec/PlistBuddy -c "Set :CFBundleShortVersionString $NEW_VERSION"
apps/mobile/ios/brandA-Info.plist

# Update Android version
# (Implement version code calculation)

echo "✅ Version bumped to $NEW_VERSION"

```

Deployment Environments

Environment	Branch	Deployment Target	Auto-deploy
Development	develop	App Center (Internal)	Yes
Staging	release/*	App Center (Beta)	Yes
Production	main	App Store / Play Store	Manual approval

Release Checklist

File: `.github/PULL_REQUEST_TEMPLATE/release.md`

```

## Release Checklist

### Pre-Release
- [ ] All tests passing
- [ ] No linting errors
- [ ] Version bumped correctly
- [ ] Changelog updated
- [ ] Release notes written
- [ ] Feature flags configured
- [ ] Environment variables updated

### Build
- [ ] Android builds successfully
- [ ] iOS builds successfully
- [ ] No build warnings

### Testing
- [ ] QA testing completed
- [ ] Regression tests passed
- [ ] Performance testing done
- [ ] Accessibility testing done

```

Deployment

- [] Staged to internal testers
- [] Beta feedback addressed
- [] App Store metadata updated
- [] Play Store metadata updated
- [] Screenshots updated

Post-Release

- [] Monitor crash reports
- [] Monitor analytics
- [] Check user feedback
- [] Hotfix plan ready

Best Practices

1. Code Organization

Modular Structure

```
// ✅ Good: Grouped exports
export const DateUtils = {
  formatDate: (date: Date) => { /* ... */ },
  isWeekend: (date: Date) => { /* ... */ },
  addDays: (date: Date, days: number) => { /* ... */ }
}

// ❌ Bad: Individual exports
export const formatDate = (date: Date) => { /* ... */ }
export const isWeekend = (date: Date) => { /* ... */ }
export const addDays = (date: Date, days: number) => { /* ... */ }
```

Barrel Exports

```
// packages/feat-loyalty/src/index.ts
export { LoyaltyScreen } from './screens/LoyaltyScreen'
export { RewardsScreen } from './screens/RewardsScreen'
export { useLoyaltyPoints } from './hooks/useLoyaltyPoints'
export type { LoyaltyState } from './models/types'
```

2. Dependency Management

Package Boundaries

```
// ✅ Good: Clear dependencies
{
  "dependencies": {
    "@packages/common": "*",
    "@packages/toolkit": "*"
  }
}
```

```

    "@packages/shared-localization": "*"
  }
}

// ❌ Bad: Circular or unclear dependencies
{
  "dependencies": {
    "@packages/feat-payments": "*", // Feature depending on another feature
    "lodash": "^4.17.21"           // Duplicated across packages
  }
}

```

3. Type Safety

Strict TypeScript Configuration

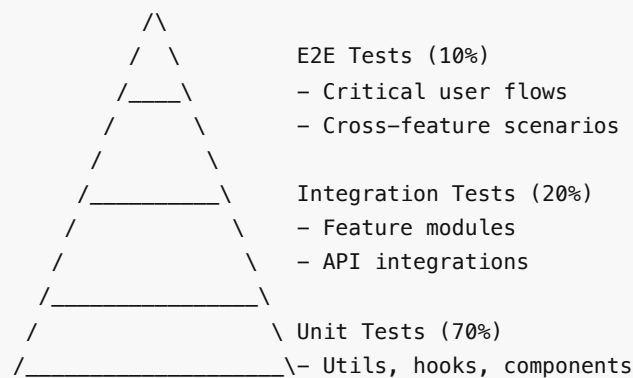
```

{
  "compilerOptions": {
    "strict": true,
    "noImplicitAny": true,
    "strictNullChecks": true,
    "strictFunctionTypes": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true
  }
}

```

4. Testing Strategy

Test Pyramid



Test Organization

```

packages/feat-loyalty/
├── src/
│   ├── components/
│   │   └── LoyaltyCard.tsx
└── __tests__/

```

```
├─ unit/
│   └─ LoyaltyCard.test.tsx
├─ integration/
│   └─ loyaltyFlow.test.tsx
└─ mocks/
    └─ loyaltyData.ts
```

5. Performance

Code Splitting

```
// Lazy load features
const LoyaltyNavigator = lazy(() =>
  import('@packages/feat-loyalty').then(m => ({ default: m.LoyaltyNavigator })))
)

// Use with Suspense
<Suspense fallback={<LoadingScreen />}>
  <LoyaltyNavigator />
</Suspense>
```

Memoization

```
// Memoize expensive calculations
const sortedItems = useMemo(() =>
  items.sort((a, b) => a.price - b.price),
  [items]
)

// Memoize callbacks
const handlePress = useCallback(() => {
  navigation.navigate('Details', { id })
}, [navigation, id])
```

6. Security

Sensitive Data

```
// ❌ Never commit secrets
const API_KEY = 'sk_live_abc123...'

// ✅ Use environment variables
const API_KEY = process.env.API_KEY

// ✅ Use secure storage
import { setGenericPassword } from 'react-native-keychain'
await setGenericPassword('user', token)
```

7. Accessibility

```
// ✅ Good: Proper accessibility labels
<TouchableOpacity
  accessible={true}
  accessibilityLabel="Add to cart"
  accessibilityHint="Double tap to add item to your cart"
  accessibilityRole="button"
>
  <Text>Add to Cart</Text>
</TouchableOpacity>
```

8. Error Handling

```
// ✅ Comprehensive error handling
try {
  const result = await fetchData()
  return result
} catch (error) {
  // Log to monitoring service
  logger.error('Failed to fetch data', { error })

  // Show user-friendly message
  showErrorToast('Unable to load data. Please try again.')

  // Return fallback
  return fallbackData
}
```

9. Documentation

```
/**
 * Calculates loyalty points based on purchase amount
 *
 * @param amount - Purchase amount in cents
 * @param tier - User's loyalty tier
 * @returns Calculated points
 *
 * @example
 * ```ts
 * const points = calculatePoints(10000, 'gold')
 * // returns 150
 * ```
 */
export const calculatePoints = (
  amount: number,
  tier: LoyaltyTier
): number => {
  // Implementation
}
```

10. Monitoring & Analytics

```
// Track feature usage
analytics.track('feature_used', {
  feature: 'loyalty',
  brand: getBrand(),
  screen: 'rewards'
})

// Monitor performance
performance.mark('loyalty_screen_start')
// ... render logic
performance.mark('loyalty_screen_end')
performance.measure('loyalty_screen_render', 'loyalty_screen_start',
'loyalty_screen_end')
```

Appendix

A. Useful Commands

```
# Setup
yarn install                # Install all dependencies
yarn pods                  # Install iOS pods

# Environment
yarn env:brandA:dev         # Setup Brand A development
yarn env:brandB:staging     # Setup Brand B staging

# Development
yarn start                 # Start Metro bundler
yarn ios:brandA             # Run Brand A on iOS
yarn android:brandB        # Run Brand B on Android

# Testing
yarn test                  # Run all tests
yarn test:watch            # Run tests in watch mode
yarn test:coverage         # Generate coverage report

# Linting
yarn lint                  # Lint all files
yarn lint:fix              # Auto-fix linting issues

# Building
yarn build:ios:brandA      # Build iOS for Brand A
yarn build:android:brandB  # Build Android for Brand B

# Cleaning
```

```
yarn clean                # Clean build artifacts
yarn clean:node            # Remove all node_modules
```

B. Recommended Tools

- **Version Control:** Git, GitHub
- **Package Manager:** Yarn Workspaces
- **State Management:** MobX, React Query
- **Navigation:** React Navigation
- **UI Components:** Custom design system
- **Testing:** Jest, React Native Testing Library, Detox
- **Linting:** ESLint, Prettier
- **Type Checking:** TypeScript
- **Design System:** Monorepo package (atomic design: atoms, molecules, organisms)
- **CI/CD:** GitHub Actions, Fastlane
- **Monitoring:** Sentry, Firebase Crashlytics
- **Analytics:** Firebase Analytics, Mixpanel
- **Feature Flags:** Firebase Remote Config, LaunchDarkly
- **Distribution:** App Center, TestFlight, Play Console

C. Folder Naming Conventions

Pattern	Purpose	Example
design-system	UI components + brand themes	design-system/src/atoms, design-system/themes/brandA
shared-*	Cross-app shared features	shared-auth, shared-profile
feat-*	Optional features	feat-loyalty, feat-payments
brand-*-*	Brand-specific features	brandA-rewards, brandB-marketplace

D. Git Branching Strategy

```
main                # Production releases
├─ release/2.5.x    # Release branches
│   └─ develop      # Development integration
│       ├── feature/loyalty-v2
│       ├── feature/new-checkout
│       └─ bugfix/payment-issue
```

E. Migration Guide

When adding a new brand to existing setup:

1. **Create theme package:** packages/theme-newBrand/
2. **Add build variants:** Android flavor + iOS scheme
3. **Create config files:** newBrand.json , environment configs
4. **Update feature matrix:** Add brand to features.ts
5. **Add CI/CD workflow:** build-newBrand.yml
6. **Create app icons:** Brand-specific assets
7. **Setup app stores:** Create listings in stores

8. **Configure secrets:** Add signing certificates
 9. **Test thoroughly:** All features + edge cases
 10. **Deploy:** Follow standard deployment process
-

Conclusion

This white-label architecture provides a robust foundation for building and maintaining multiple branded applications from a single codebase. Key takeaways:

- **Scalability:** Easy to add new brands and features
- **Maintainability:** Shared code reduces duplication
- **Flexibility:** Brand-specific customization where needed
- **Quality:** Centralized testing and deployment
- **Efficiency:** Single team, multiple products

For questions or contributions, please refer to the project's contribution guidelines.
